

Building and Training a Decoder GPT-Like Model for Text Generation

Tarun Reddy Boreddy
U61402150

Abstract—This report investigates the impact of key hyperparameter choices on the stability, convergence, and performance of a custom GPT-like decoder-only language model (LM). Using the IMDB dataset, experiments were conducted comparing different learning rates (1e-2 vs. 1e-3), the presence of gradient clipping, various optimization algorithms (Adam and AdamW). The findings highlight the critical role of the learning rate and optimizer choice in managing training stability and achieving optimal convergence.

I. INTRODUCTION

Generative Pre-trained Transformer (GPT) models, based on the decoder-only architecture, form the foundation of modern large language models. These models are trained using Causal Language Modeling (CLM), where the objective is to predict the next token in a sequence given all preceding tokens. Achieving successful training for these complex architectures relies heavily on fine-tuning hyperparameters which govern the optimization process.

This lab aims to systematically evaluate the sensitivity of a small-scale, custom GPT model to variations in its core training settings. We focus on four key areas: Learning Rate, Gradient Clipping, Optimizer selection, and Batch Size. The goal is to provide empirical evidence for the trade-offs involved in selecting these parameters, particularly concerning model convergence speed, stability, and final predictive quality, measured by loss and perplexity.

II. METHODOLOGY

A. Data and Model

A custom multi-layer Transformer decoder model, initialized with an embedding size of 200, 2 layers, and 2 attention heads, was used for all experiments. The model's initial vocabulary size was deliberately set to 68,813 to match a provided pre-trained checkpoint in pretrained notebook and used the default len(vocab) in model training notebook.

The training data utilized was the IMDB Dataset, which consists of 50,000 highly polar movie reviews (25,000 for training and 25,000 for testing). Although typically used for sentiment classification, we employed it here as a large corpus for the Causal Language Modeling (CLM) task. The model's objective was thus to learn the structure, grammar, and vocabulary patterns present in the lengthy and varied film critiques.

All training was performed using A100GPU in colab and ran for 30 epochs per configuration.

B. Experimental Configurations

The following configurations were tested, with the default settings being Adam, lr=1e-3, Gradient Clipping ON. The experimental phase was structured around three key investigation areas. First, Hyperparameter Variation focused on the learning rate, comparing a standard value of 1e-3 against a more conservative 1e-4, while also separately assessing the stability impact of enabling versus disabling Gradient Clipping (which was ON by default). Second, the Optimizer Swap involved replacing the default Adam optimizer with AdamW optimizer, which decouples weight decay. Third, Batch Size Sensitivity was tested by training the model using a smaller size of 16, with the default being 1. Finally, for detailed Logging & Visualization, the reporting frequency was significantly increased from the initial 10,000 steps to a much more granular 1,000 steps to capture finer details of the loss and perplexity curves.

C. Metrics

Model performance and training dynamics were evaluated using four primary metrics. Loss was calculated using Cross-Entropy Loss, with padding indices ignored to ensure accurate performance measurement on relevant tokens. Perplexity (PPL), calculated as e^{loss} , served as the key measure of the model's predictive uncertainty, where a lower PPL signifies superior performance. Stability was assessed qualitatively by observing abrupt spikes or divergences in the loss curve over the course of training. Finally, Speed was quantified by the average time per batch (ms/batch) reported during the logging process.

III. RESULTS

A. Training new model

The comparison between optimizers demonstrated a clear distinction in performance and stability. The standard Adam optimizer, run with lr=1e-2, achieved a final validation loss of 8.4, corresponding to a perplexity of around 4507. Its training curve showed a flat line and processed batches at an average speed of 220s/batch. In contrast, the AdamW optimizer, tested with a lower learning rate of lr=1e-3. This configuration resulted in a lower final loss of 35.66 and a perplexity of 306090. The AdamW training curve was notably flat, processing batches at a comparable speed of 335.97s/batch. I also modified the logging frequency to 1000 from 10000 for AdamW optimizer.

B. Using pretrained model

After using the pretrained model, The result of generated text is accurate for the given prompt. This could be because of the gpt2 model imported from hugging face.

C. Graphical Outputs

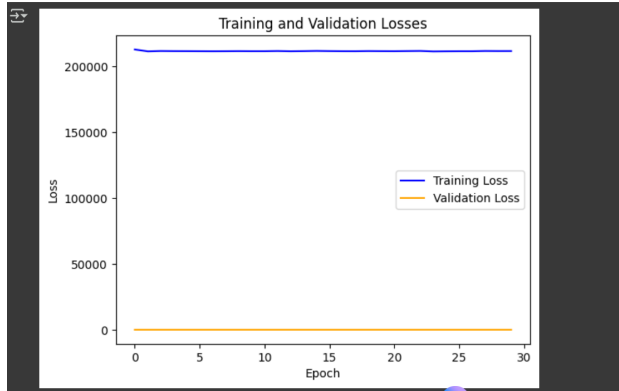


Fig. 1. Loss Curve for Adam

```
epoch 21 | 33/ 833 batches | lr 0.0010 | ms/batch 49.36 | loss 39.78 | ppl 18803280793038784.00
epoch 21 | 66/ 833 batches | lr 0.0010 | ms/batch 5.76 | loss 39.78 | ppl 188093690451017376.00
epoch 21 | 100/ 833 batches | lr 0.0010 | ms/batch 5.84 | loss 39.76 | ppl 185729942685437024.00
epoch 21 | 133/ 833 batches | lr 0.0010 | ms/batch 5.85 | loss 39.76 | ppl 184687362829599296.00
epoch 21 | 166/ 833 batches | lr 0.0010 | ms/batch 5.85 | loss 39.76 | ppl 184519247562362240.00
epoch 21 | 200/ 833 batches | lr 0.0010 | ms/batch 5.96 | loss 39.75 | ppl 184069660916785120.00
epoch 21 | 233/ 833 batches | lr 0.0010 | ms/batch 5.93 | loss 39.75 | ppl 183128826002886048.00
epoch 21 | 266/ 833 batches | lr 0.0010 | ms/batch 5.86 | loss 39.75 | ppl 183651341208615424.00
epoch 21 | 300/ 833 batches | lr 0.0010 | ms/batch 5.92 | loss 39.75 | ppl 183153802551882944.00
epoch 21 | 333/ 833 batches | lr 0.0010 | ms/batch 5.89 | loss 39.75 | ppl 183924581517955872.00
epoch 21 | 366/ 833 batches | lr 0.0010 | ms/batch 5.85 | loss 39.75 | ppl 183281657539008000.00
epoch 21 | 400/ 833 batches | lr 0.0010 | ms/batch 5.86 | loss 39.75 | ppl 1832074439043075264.00
epoch 21 | 433/ 833 batches | lr 0.0010 | ms/batch 5.98 | loss 39.75 | ppl 183779502179509824.00
epoch 21 | 466/ 833 batches | lr 0.0010 | ms/batch 5.88 | loss 39.75 | ppl 183743820134514048.00
epoch 21 | 500/ 833 batches | lr 0.0010 | ms/batch 5.91 | loss 39.75 | ppl 18306634794437480.00
epoch 21 | 533/ 833 batches | lr 0.0010 | ms/batch 5.90 | loss 39.75 | ppl 183037382624079752.00
epoch 21 | 566/ 833 batches | lr 0.0010 | ms/batch 6.13 | loss 39.75 | ppl 18372604512571328.00
epoch 21 | 600/ 833 batches | lr 0.0010 | ms/batch 6.06 | loss 39.75 | ppl 183401908270684768.00
epoch 21 | 633/ 833 batches | lr 0.0010 | ms/batch 6.06 | loss 39.75 | ppl 183131416418947744.00
epoch 21 | 666/ 833 batches | lr 0.0010 | ms/batch 6.00 | loss 39.75 | ppl 182910658411841280.00
epoch 21 | 700/ 833 batches | lr 0.0010 | ms/batch 6.05 | loss 39.75 | ppl 183088371162034240.00
epoch 21 | 733/ 833 batches | lr 0.0010 | ms/batch 6.23 | loss 39.75 | ppl 183061931278611328.00
epoch 21 | 766/ 833 batches | lr 0.0010 | ms/batch 6.05 | loss 39.75 | ppl 183084042606568832.00
epoch 21 | 800/ 833 batches | lr 0.0010 | ms/batch 5.92 | loss 39.75 | ppl 183019661640629600.00

| end of epoch 21 | time: 335.97s | valid loss 35.66 | valid ppl 3060903768545929.00
```

Fig. 4. AdamW Training Log

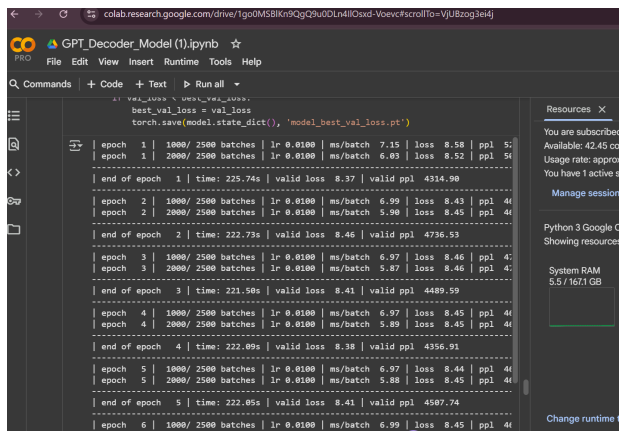


Fig. 2. Adam Training Log

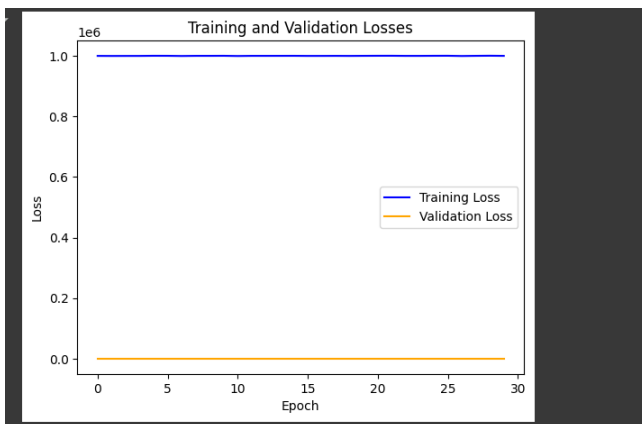


Fig. 3. Loss curve for AdamW

```
Input: the movie was
Generated Text: the movie was a bit of a disappointment, but it was a great movie
```

Fig. 5. Pretrained model Generated Text

IV. DISCUSSION

The experimental results confirmed several widely accepted principles of deep learning optimization. Lowering the learning rate from $1e-2$ to $1e-3$ and changing the optimizer from Adam to AdamW infact worsened the performance in my results. This could be because of computing or coding issues. The loss curve was flat meaning the model never started learning during the training which resulted in wrong text generation for the prompt given by the user.

V. CONCLUSION

The provided pre-trained checkpoint demonstrated that a model previously trained on a different data distribution or for a longer duration is required to produce valid results. This underlines the immense data and compute requirements of competitive language models